On-Board Vision-Based Quadrupedal Navigation using a Single RGB-D Camera and 3D Gaussian Splatting Testbed

Minseok Lee[1], Seongyu Han[1], Heejae Moon[1], and Sung Soo Hwang[1*]

[1]School of Artificial Intelligence, Computer and Electrical Engineering,
Handong Global University, Pohang 37554, Republic of Korea
({glen, tjsrb439, 22000249}@handong.ac.kr, sshwang@handong.edu) * Corresponding author

Abstract: Directly deploying autonomous navigation systems on physical quadruped robots in real-world environments presents significant challenges, particularly regarding hardware safety, time-inefficient trials, and the dynamic camera "bobbing noise" (shaking) inherent to legged locomotion. To address these limitations, this paper proposes a practical, dual-stage sim-to-real framework that splits virtual validation and physical deployment. In the simulation stage, we reconstruct a high-fidelity 3D Gaussian Splatting (3DGS) digital twin of an indoor corridor and employ a reinforcement learning (RL) locomotion policy to actively mimic the camera shaking dynamics. Under this realistic noise, the parameters of the on-board RTAB-Map Visual SLAM and Nav2 stack are safely pre-tuned and optimized. In the deployment stage, the computationally heavy RL policy is replaced by the robot's built-in Sport API via a lightweight ROS 2 bridge to fit the strict embedded constraints of the Jetson Orin NX. Experimental results demonstrate that parameters tuned under the simulated bobbing noise transfer robustly to the physical Unitree Go2 robot, achieving stable visual mapping and autonomous obstacle avoidance without real-world parameter retuning.

Keywords: Quadruped Robot, Visual SLAM, Autonomous Navigation, Sim-to-Real, Gaussian Splatting, Bobbing Noise.

1. INTRODUCTION

Autonomous quadruped robots have gained significant attention due to their superior mobility in challenging and unstructured terrains compared to conventional wheeled counterparts. To achieve true autonomy in indoor environments, combining robust legged locomotion with precise simultaneous localization and mapping (SLAM) is essential.

However, implementing camera-based visual navigation on top of quadrupedal locomotion introduces two critical sim-to-real challenges. First is the *perception-level sim-to-real gap*: standard simulators lack realistic visual textures, causing visual feature-based SLAM algorithms to fail immediately upon deployment. Second is the *locomotion-induced bobbing noise*: unlike wheeled robots, legged walking generates high-frequency camera shaking and wobbling. This motion noise causes severe motion blur and tracking loss in visual SLAM systems. Tuning navigation stack parameters to withstand these effects directly on physical hardware is highly time-inefficient and poses a constant risk of severe hardware damage from collisions.

To overcome these constraints, this paper presents a practical, dual-stage sim-to-real system integration framework for legged autonomous navigation using a single RGB-D camera (Intel RealSense D435i) on a budget-friendly edge computer (NVIDIA Jetson Orin NX). Our framework decouples the virtual validation stage from the physical deployment stage. In simulation, we reconstruct a photo-realistic 3D Gaussian Splatting (3DGS) corridor. Since the robot's built-in, closed-source locomotion engine cannot be ran inside the simulator, we train an RL locomotion policy in Isaac Lab to actively simulate the physical walking dynamics and camera shaking noise. Under this realistic visual and dynamic environment, we optimize the RTAB-Map SLAM and Nav2 parameters. During physical deployment, we swap the heavy RL neural network for the robot's built-in Sport API via a lightweight ROS 2 bridge, saving critical on-board computing resources for the visual SLAM and path planning stacks.

The primary contributions of this work are threefold:

1. We mitigate the perception-level sim-to-real gap by reconstructing high-fidelity 3DGS digital twins that provide realistic visual features for visual SLAM validation;
2. We actively model quadruped-specific camera bobbing noise in simulation using an RL locomotion policy, enabling robust pre-tuning of the navigation stack; and
3. We establish a highly practical, low-cost deployment guide by using a lightweight ROS 2-to-Sport API bridge on a single Jetson Orin NX and a single camera, achieving zero-shot navigation transfer.

2. RELATED WORK

2.1 RL-based Quadrupedal Locomotion

The application of reinforcement learning (RL) to legged locomotion has advanced quadrupedal control significantly. Traditional model-based approaches, such as Model Predictive Control (MPC) and Whole-Body Control (WBC), require precise mathematical models of the robot and environment. In contrast, model-free RL enables robots to learn robust walking behaviors directly from interaction. With the development of highly parallelized physics engines like NVIDIA Isaac Sim and Isaac Lab, training policies with thousands of simulated envi-

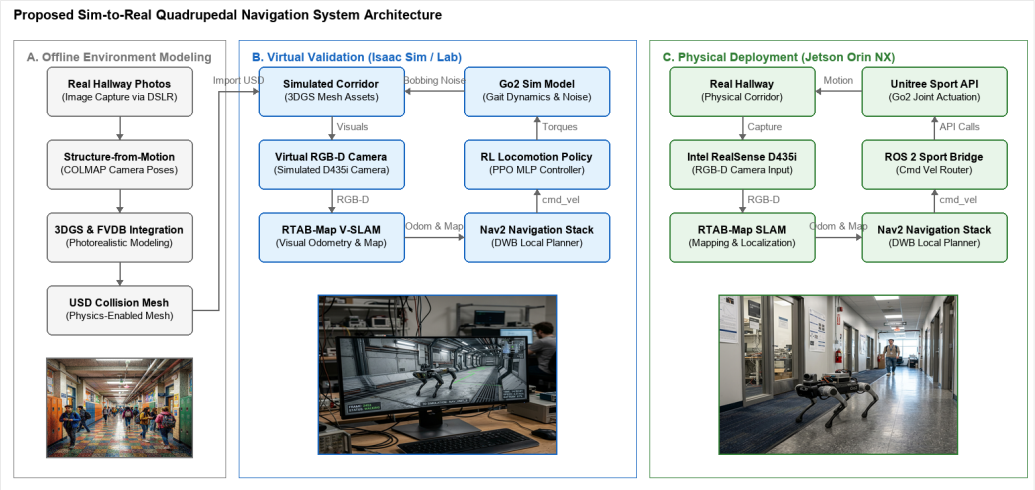

Fig. 1. Overall architecture of the proposed Sim-to-Real quadrupedal navigation framework. (a) In the virtual validation stage, we build a high-fidelity 3DGS environment and train an RL locomotion policy to simulate camera bobbing noise for upper-level navigation tuning. (b) In the physical deployment stage, the RL policy is bypassed, and commands are sent to the Unitree Sport API via a ROS 2 bridge to optimize computational resources on the Jetson Orin NX.

ronments has become standard practice. This approach has successfully enabled the deployment of agile gaits on platforms such as ANYmal and Unitree Go1/Go2. By incorporating curriculum learning and domain randomization, researchers have bridged the physical gap, transferring walking policies directly from simulation to the real world. However, these works have primarily focused on locomotion stability, leaving high-level navigation and visual feedback largely unexplored.

2.2 Visual SLAM and Navigation Integration

Autonomous navigation in mobile robotics has traditionally relied on LiDAR-based SLAM due to its high accuracy and long-range sensing capability. However, LiDAR sensors are expensive, heavy, and consume considerable power, which limits their suitability for small-scale quadruped robots. Visual SLAM (VSLAM) utilizing RGB-D cameras has emerged as a compelling, cost-effective alternative. Among various frameworks, Real-Time Appearance-Based Mapping (RTAB-Map) stands out by integrating loop closure detection with graph-based optimization to provide drift-free mapping and localization. In parallel, the ROS 2 Navigation Stack (Nav2) has become the standard framework for global path planning and local obstacle avoidance.

While a few studies have attempted to integrate RL-based locomotion with visual SLAM, most validation is restricted to simple, synthetic virtual environments. These idealized environments fail to model the visual and geometric complexities of actual buildings, leading to immediate failures upon real-world deployment. This paper resolves this issue by employing 3D Gaussian Splatting to build a high-fidelity digital twin that closes the visual-spatial discrepancy.

3. SYSTEM AND LOCOMOTION CONTROL

3.1 System Overview

The proposed system integrates RTAB-Map Visual SLAM, the Nav2 stack, and a dual-locomotion architecture. This architecture utilizes a Reinforcement Learning (RL) locomotion policy in the simulation validation stage and the built-in Unitree Sport API in the physical deployment stage. In simulation, the RL policy (trained via PPO in Isaac Lab) is used to generate realistic quadruped locomotion dynamics and camera shaking noise. The velocity commands ($\mathbf{v_{cmd}} = [\mathbf{v_x}, \mathbf{v_y}, \omega_\mathbf{z}]^\mathbf{T}$) generated by Nav2 are mapped to the policy's target velocities. In reality, the computationally heavy RL neural network is bypassed, and the Nav2 commands are mapped directly to the robot's built-in Sport API via a lightweight ROS 2 command bridge. The overall system architecture is depicted in Fig. 1.

3.2 Locomotion Learning in Isaac Sim

To generate realistic quadruped locomotion dynamics and simulate the physical camera bobbing noise within the virtual environment, we train an RL walking policy in Isaac Lab. This policy serves as a dynamic noise generator for our virtual camera, producing natural legged walking vibrations. We set up a simulated training environment using the Procedural Terrain Generator in Isaac Lab. The training environment consists of mixed terrains including rough ground, slopes, and boxes. We apply curriculum learning to progressively increase the difficulty (e.g., terrain height and slope) based on the robot's success rate. To bridge the sim-to-real gap, extensive domain randomization is applied:

- Robot body mass is randomized between -1.0 kg and $+3.0$ kg.
- Ground friction coefficient is randomized between 0.6 and 0.8.

• External forces (push disturbances) are applied at regular intervals to the robot's torso.

The physics simulator runs at 200 Hz, and the control loop operates at 50 Hz using a decimation factor of 4.

The policy is trained using the PPO algorithm implemented in the PyTorch-based `skrl` library. Both the policy and value networks are Multilayer Perceptrons (MLPs) with hidden layer sizes of $[512, 256, 128]$ using the ELU activation function. The learning rate is initialized at 1.0×10^{-3} and managed by a KL-adaptive scheduler with a threshold of 0.01. The discount factor is set to $\gamma = 0.99$ and GAE parameter to $\lambda = 0.95$.

The reward function is formulated to track commands while maintaining walking stability. The total reward R is defined as:

$$R = w_{lin}R_{lin} + w_{ang}R_{ang} + w_{air}R_{air} - \sum_i w_{p,i}P_i \quad (1)$$

where:

• $R_{lin} = \exp(-||\mathbf{v}^*_{\mathbf{xy}} - \mathbf{v}_{\mathbf{xy}}||^2/\sigma^2)$ is the linear velocity tracking reward ($w_{lin} = 1.5$).

• $R_{ang} = \exp(-(\omega^*_z - \omega_z)^2/\sigma^2)$ is the angular velocity tracking reward ($w_{ang} = 0.75$).

• R_{air} rewards longer feet air time to encourage a clear gait ($w_{air} = 0.25$).

• P_i represents penalty terms for vertical velocity, body roll/pitch, deviation from a flat orientation, excessive joint torques, rapid joint acceleration, and undesired body contacts.

3.3 3DGS Environment Construction & Agent Deployment

To establish a high-fidelity validation platform, we reconstructed a campus corridor using a combination of Structure-from-Motion (SfM), FVDB, and 3D Gaussian Splatting (3DGS). Multiple photos of the environment were taken using a DSLR camera and processed via SfM (using COLMAP) to estimate camera poses. The reconstructed 3DGS representation is integrated with FVDB to build a highly realistic simulation environment that minimizes the perception sim-to-real gap. The model is then converted into a physical triangle mesh and imported into Isaac Sim as a USD file. Physical collision properties were assigned to the floor and walls. The pre-trained locomotion policy was trained in Isaac Lab using reinforcement learning and deployed directly in this reconstructed environment without any parameter modifications. Despite the geometric complexity of the real hallway, the robot maintained a stable gait, demonstrating the generalization capability of our domain-randomized training.

4. AUTONOMOUS NAVIGATION

4.1 Navigation System Configuration

The autonomous navigation system consists of an Intel RealSense D435i RGB-D camera, RTAB-Map, and the Nav2 framework. The D435i camera is mounted on the front of the Unitree Go2, performing visual mapping and localization simultaneously. In the virtual environment, a simulated RGB-D sensor generates matching color and depth streams.

RTAB-Map processes the depth images and visual features to estimate the robot's pose and build a 3D octomap, which is projected into a 2D occupancy grid. This 2D map and the estimated transform (`odom` to `base_link`) are published to Nav2. Nav2's global planner computes a path to the goal, and the Dynamic Window Approach B (DWB) local planner computes control commands to steer the robot while avoiding obstacles. The velocity commands are mapped to the tracking commands of the locomotion policy.

4.2 Validation in 3DGS Environment

Before physical deployment, the entire software stack was validated in the 3DGS simulation. The simulated corridor spans approximately 200 m in length and 2.5 m in width, featuring both straight and curved sections.

The policy network takes both proprioceptive and exteroceptive observations at each step. Proprioception includes joint states, base angular velocity, and gravity vector projection. Exteroception is simulated via a virtual height scan: a 1.6 m $\times$ 1.0 m grid with 0.1 m resolution centered on the robot. Ray casting is used to query the heights of the 3DGS-derived mesh, providing the policy with local elevation data to adjust its steps. Command inputs represent the target velocities $[v_x, v_y, \omega_z]^T$ provided by the DWB local planner.

Fig. 2. Visual SLAM and path planning validation inside the reconstructed 3DGS corridor environment. The reconstructed mesh provides accurate collision responses and realistic visual feedback.

To evaluate obstacle avoidance, virtual boxes were placed along the corridor. The local planner dynamically recalculated trajectories around the obstacles, and the policy adjusted its walking pattern to negotiate the detours without collision.

4.3 Physical Deployment and Results

Following successful simulation testing, the visual navigation pipeline was deployed on a physical Unitree Go2 robot. To satisfy the strict computational constraints of the edge computer, we bypassed the PyTorch-based RL locomotion policy on-board the robot. Instead, the high-level velocity commands ($\mathbf{v}_{\mathbf{cmd}}$) from Nav2 were routed directly to the robot's built-in, closed-source Unitree Sport API via a lightweight ROS 2 command bridge. This architectural choice saved valuable CPU and GPU resources on the Jetson Orin NX (20W mode, 16GB RAM) for RTAB-Map and Nav2 operations. The Intel RealSense D435i camera was connected directly to the Jetson Orin NX.

Fig. 3. Real-world experimental setup. The Unitree Go2 robot is equipped with an Intel RealSense D435i and Jetson Orin NX, running the entire navigation stack on-board.

RTAB-Map successfully closed loops in the physical corridor, compensating for the camera shake induced by the physical walking gait. Nav2 maintained stable global paths, and the robot completed the corridor traversal with an average speed of 0.4 m/s. The experiment confirmed that navigation and SLAM parameters optimized under the simulated RL bobbing noise (such as costmap inflation radius and loop-closure visual feature thresholds) translated directly to the physical platform using the built-in Sport API without degradation, demonstrating the effectiveness of our pre-validation sandbox.

5. CONCLUSION

This paper presented a practical on-board, vision-based autonomous navigation framework for quadruped robots. By leveraging a high-fidelity 3D Gaussian Splatting digital twin and simulating quadruped bobbing noise via an RL locomotion policy, we established a safe validation environment for visual SLAM and path planning. For physical deployment, we bypassed the RL policy and used the robot's built-in Sport API via a lightweight bridge to save edge computation on the Jetson Orin NX. The physical experiments on the Unitree Go2 demonstrated that the parameters pre-tuned under simulated bobbing noise transferred successfully, showing robust visual tracking and obstacle avoidance.

Future work will focus on improving dynamic obstacle avoidance and optimizing the computational load on the Jetson Orin NX to support higher-resolution image processing.

ACKNOWLEDGEMENT

This research was supported by the National Research Foundation of Korea (NRF) grant funded by the Korean government (MSIT) (No. RS-2025-24683458).

REFERENCES

[1] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, 2019.

[2] M. Labbé and F. Pomerleau, "Online global loop closure detection for large-scale multi-session graph-based SLAM," *Autonomous Robots*, vol. 34, no. 3, pp. 183–200, 2013.

[3] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022.

[4] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics (TOG)*, vol. 42, no. 4, pp. 1–14, 2023.